

NAME :FAITH GITAH

REG NO: C027-01-0889/2024

Question 1: design decisions (2 marks)

(i) What is your admission number, and how did it determine your dataset (number of gigs, categories, and currency)?

My Admission Number is C027-01-0889/2024 .

The last digit is **9**, so I created **14 gigs** ($5 + 9 = 14$).

The number **0889** is **odd**, so i assigned my categories:

- Marketing
- Data
- Consulting

The first two digits are **08**, which is less than 10, so I used **KES** as the currency for all the gig budgets.

(ii) Why did you choose to include the status field in your gig data model? What are the possible statuses and what do they represent?

It shows the gig status. It also helps both clients and freelancers keep track of the progress of a job.

1. **Open** - the gig is available for freelancers to take.
2. **In Progress** - the gig is currently being worked on.
3. **Closed** - the gig is finished or no longer available.

Question 2: Validation (2 marks)

(i) What validation rules did you apply to the budget field in your Pydantic model? Why is this validation important?

$gt=0$, this ensures that no gig can be created with a negative or zero budget. It's important because a valid project must have a positive payment amount.

(ii) How did you ensure that the category field only accepts values from your assigned category list?

I used `Literal["Marketing", "Data", "Consulting"]`

This ensures that only the assigned categories can be entered.

Question 3: Error handling (2 marks)

- (i) **What status code does your API return when a user tries to retrieve a gig that does not exist? Explain why this is the appropriate status code.**

The API returns 404 Not Found. It tells the user that the requested gig does not exist in the database.

- (ii) **What status code does your API return when a user tries to create a gig with invalid data? Explain why**

API returns 422 Unprocessable Entity .If the data doesn't meet the validation rules , the request is rejected with a 422 status code.

Question 4: search and filtering (2 marks)

- (i) **Describe how your `/gigs/search` endpoint works. How does it handle searches where the query string does not match any gig?**

The `/gigs/search` endpoint searches for gigs using the title. The user enters a search word through the `q` query parameter.

The endpoint compares the search word with each gig title without considering uppercase or lowercase letters. If no gig matches the search word, the endpoint returns an empty list(`[]`)

- (ii) **Describe how your `/gigs` endpoint with filtering works. What are the optional query parameters, and how do they work?**

The `/gigs` endpoint returns all gigs by default.

- 1.**category** – returns gigs in the selected category.
2. **min_budget** – returns gigs with a budget greater than or equal to the given amount.
- 3.**max_budget** – returns gigs with a budget less than or equal to the given amount.

Question 5: Code walkthrough (2 marks)

- (i) **Write a brief explanation of the PUT /gigs/{gig_id} endpoint. What does it do, and how does it handle a request to update a gig that does not exist?**

The PUT /gigs/{gig_id} endpoint is used to update an existing gig. It allows the user to change the budget, the status, or both.

The endpoint first checks whether the gig exists. If it does not exist, it returns a 404 Not Found error. If the gig is found, the provided values are updated and the updated gig is returned.

- (ii) **Write a brief explanation of the POST /gigs endpoint. What does it do, and how does it ensure that the new gig is created correctly?**

The POST /gigs endpoint is used to create a new gig.

It receives the gig details from the user and validates them using the GigCreate Pydantic model. If the data is valid, a new ID is generated automatically, the currency is set to KES, and the status is set to Open. The new gig is then added to the list and returned as the response.